

Clases

```
public <abstract> class NombreClaseHija extends NombreClasePadre {
    private int atributo1;
    private String atributo2;
    public NombreClase(int atributo1, String atributo2) {
        this.atributo1 = atributo1;
        this.atributo2 = atributo2;
    }
    public int getAtributo1() {
        return atributo1;
    }
    public void setAtributo1(int atributo1) {
        this.atributo1 = atributo1;
    }
    @Override
    public String toString() {
        return "NombreClase{" +
            "atributo1=" + atributo1 +
            ", atributo2='" + atributo2 + '\'' +
            '}';
    }
}
```

Bases de datos

Conexión BD MySQL

```
public class DBConnection {
    private static final String URL =
"jdbc:mysql://localhost:3306/bd";
    private static final String USER = "root";
    private static final String PASSWORD = "Password1234";
```

```
    public static Connection getConnection() throws
SQLException {
        return DriverManager.getConnection(URL, USER,
PASSWORD);
    }
}-----
```

PreparedStatement Select

```
String sql = "SELECT * FROM producto WHERE id_producto = ?";
PreparedStatement ps = con.prepareStatement(sql);
ps.setInt(1, 3);
-----
```

ResultSet

```
ResultSet rs = ps.executeQuery();
```

```
while (rs.next()) {
    int id = rs.getInt("id_producto");
    String nombre = rs.getString("nombre");
}-----
```

PreparedStatement Insert/Update/Delete

```
String sql = "INSERT INTO producto VALUES (?, ?)";
PreparedStatement ps = con.prepareStatement(sql);
ps.setInt(1, 11);
ps.setString(2, "Tablet");
s = ps.executeUpdate();
```

Excepciones

```
try {
    //codigo
} catch (TipoExcepcion1 e) {
    System.out.println("Error" + e.getMessage());
} catch (TipoExcepcion2 e) {
    System.out.println("Error" + e.getMessage());
} catch (TipoExcepcionGeneral e) {
    System.out.println("Error" + e.getMessage());
}

-----

try (TipoRecurso nombreRecurso1 = new TipoRecurso(...);
    TipoRecurso nombreRecurso2 = new TipoRecurso(...)) {
    // código que usa el recurso
} catch (TipoExcepcion e) {
    // tratamiento del error
}

-----

public class NombreExcepcionPersonalizada extends Exception {
    public ValorNoValidoException(String mensaje) {
        super(mensaje);
    }
}

-----

EjLanzar excepción: throw new NombreExcepcion("Mensaje de error");
```

Bases de datos

```
<dependency>
    <groupId>mysql</groupId>
    <artifactId>mysql-
connector-j</artifactId>

<version>8.0.33</version>
</dependency>

import java.sql.*;
Connection con =
DriverManager.getConnection(

"jdbc:mysql://localhost:3306
/tienda",
    "root",
    "password"
```

Bases de datos

Estructura general

```
Connection con = null;
PreparedStatement ps = null;
ResultSet rs = null;

try {
    con = DriverManager.getConnection(url,
usuario, password);
    String sql = "...";
    ps = con.prepareStatement(sql);
    // ps.setXxx(...)
    rs = ps.executeQuery(); // o executeUpdate()
} catch (SQLException e) {
    System.out.println(e.getMessage());
} finally {
    try {
        if (rs != null) rs.close();
        if (ps != null) ps.close();
        if (con != null) con.close();
    } catch (SQLException e) {
        System.out.println("Error al cerrar");
    }
}
```

Bases de datos

Métodos importantes

```
ps.setInt(...) ps.setString(...) ps.setDouble(...)
rs.getInt(...) rs.getString(...) rs.getDouble(...)
-----
ps.executeQuery(); // SELECT
ps.executeUpdate(); //INSERT, UPDATE, DELETE
-----
rs.close();
ps.close();
con.close();
-----try-with-resources-JDBC-----
try (Connection conexion =
DriverManager.getConnection(url, usuario, password);
    PreparedStatement ps =
conexion.prepareStatement("SELECT * FROM producto");
    ResultSet rs = ps.executeQuery()) {
    while (rs.next()) {
        System.out.println(rs.getString("nombre"));
    }
} catch (SQLException e) {
    System.out.println("Error: " + e.getMessage());
}
```

Colecciones ArrayList

```
ArrayList<Tipo> lista = new ArrayList<>();
---- Recorrer lista con for clásico-----
for (int i = 0; i < lista.size(); i++) {
    System.out.println(lista.get(i));
}
-----Recorrer lista con for-each -----
for (String elemento : lista) {
    System.out.println(elemento);
}
----- Recorrer lista con iterator-----
Iterator<String> it = lista.iterator();

while (it.hasNext()) {
    String elemento = it.next();
    System.out.println(elemento);
}
----- Métodos más importantes -----
lista.add(elemento);
lista.get(posicion);
lista.set(posicion, nuevoValor);
lista.remove(posicion);
lista.size();
lista.isEmpty();
lista.clear()
```

Serializacion

```
Implements de la clase a serializar
public class MiClase implements Serializable {
}
--Guardar objeto serializable---
ObjectOutputStream oos = null;
try {
    oos = new ObjectOutputStream(new
FileOutputStream("rutaFichero"));
    oos.writeObject(objeto);
} catch (IOException e) {
    System.out.println("Error al serializar");
}
----- Leer objeto serializable-----
ObjectInputStream ois = null;
try {
    ois = new ObjectInputStream(new FileInputStream("rutaFichero"));
    TipoObjeto objeto = (TipoObjeto) ois.readObject();
} catch (IOException e) {
    System.out.println("Error al deserializar");
} catch (ClassNotFoundException e) {
    System.out.println("No se encuentra la clase");
}
```

Colecciones HashMap

- guarda pares **clave-valor**
- **no garantiza el orden** de los elementos
- al recorrerlo, los elementos salir en cualquier orden

```
-----
HashMap<Integer, String> mapa = new HashMap<>();
----- Recorrer las claves y valores-----
----
for (Map.Entry<Integer, String> entrada :
mapa.entrySet()) {
    Integer clave = entrada.getKey();
    String valor = entrada.getValue();
    System.out.println(clave + " -> " + valor);
}
----- Recorrer con Iterator-----
Iterator<Map.Entry<Integer, String>> it =
mapa.entrySet().iterator();
-----Métodos importantes-----
mapa.put(1, "Uno"); mapa.put(2, "Dos");
String valor = mapa.get(1);
if (mapa.containsKey(1)) {
    System.out.println("La clave existe");
}
```

Colecciones LinkedHashMap

- también guarda pares clave-valor
- sí mantiene el orden de inserción
- al recorrerlo, elementos salen en el mismo orden en que se añadieron

```
-----
LinkedHashMap<Integer, String> mapa = new LinkedHashMap<>();
-----Recorrer clave y valor -----
for (Map.Entry<Integer, String> entrada : mapa.entrySet()) {
    Integer clave = entrada.getKey();
    String valor = entrada.getValue();

    System.out.println(clave + " -> " + valor);
}
----- Recorrer iterator-----
Iterator<Map.Entry<Integer, String>> it =
mapa.entrySet().iterator();

while (it.hasNext()) {
    Map.Entry<Integer, String> entrada = it.next();
    Integer clave = entrada.getKey();
    String valor = entrada.getValue();

    System.out.println(clave + " -> " + valor);
}
```

Escritura/Lectura fichers

```
----- Escritura try-with resources---
try (BufferedWriter bw = new BufferedWriter(new FileWriter("files/productos.txt"))) {
    bw.write("Hola");
    bw.newLine();
} catch (IOException e) {
    System.out.println("Error al escribir");
}
----- Lectura try-with resources---
try (BufferedReader br = new BufferedReader(new FileReader("files/productos.txt"))) {
    String linea;

    while ((linea = br.readLine()) != null) {
        System.out.println(linea);
    }
} catch (IOException e) {
    System.out.println("Error al leer");
}
```

LECTURA POR CONSOLA

```
----- LECTURA CON BufferedReader-----
import java.io.BufferedReader;
import java.io.IOException;
import java.io.InputStreamReader;
BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
String nombre = br.readLine();
int numero = Integer.parseInt(br.readLine());
double precio = Double.parseDouble(br.readLine());
----- LECTURA CON Scanner-----
import java.util.Scanner;
Scanner sc = new Scanner(System.in);
String palabra = sc.next();
int numero = sc.nextInt();
double precio = sc.nextDouble();
boolean correcto = sc.nextBoolean();
if (sc.hasNextInt()) {
    int numero = sc.nextInt();
} else {
    System.out.println("No es un entero");
}
```

ESCRITURA CONSOLA

```
System.out.printf("Nombre: %s, Edad: %d\n", nombre, edad);
%-20s CADENA, ANCHO 20, ALINEADO IZQ
%10.2f DECIMAL, ANCHO 10, 2 DECIMALES
%8d ENTERO ANCHO 8
```

SINTAXIS BÁSICA

```
-----CONSTANTES-----
final Tipo NOMBRE_CONSTANTE = valor;
-----ESTRUCTURAS SELECTIVAS IF-ELSE--
if (condicion1) {
    // bloque 1
} else if (condicion2) {
    // bloque 2
} else {
    // bloque final
}
-----
```

SINTAXIS BASICA: BUCLES

```
do {
    // instrucciones
} while (condicion);
-----
while (condicion) {
    // instrucciones
}
-----
for (inicializacion; condicion; incremento) {
    // instrucciones
}
```

PLANTILLA RAPIDA

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int opcion;

        do {
            opcion = sc.nextInt();
            sc.nextLine();

            switch(opcion) {
                case 1:
                    break;
                case 2:
                    break;
            }

        } while(opcion != 0);

        sc.close();
    }
}
```